

Session 1: SQL Editor → New SQL Script

Session 2: SQL Editor → New SQL Script

1. Create employee table

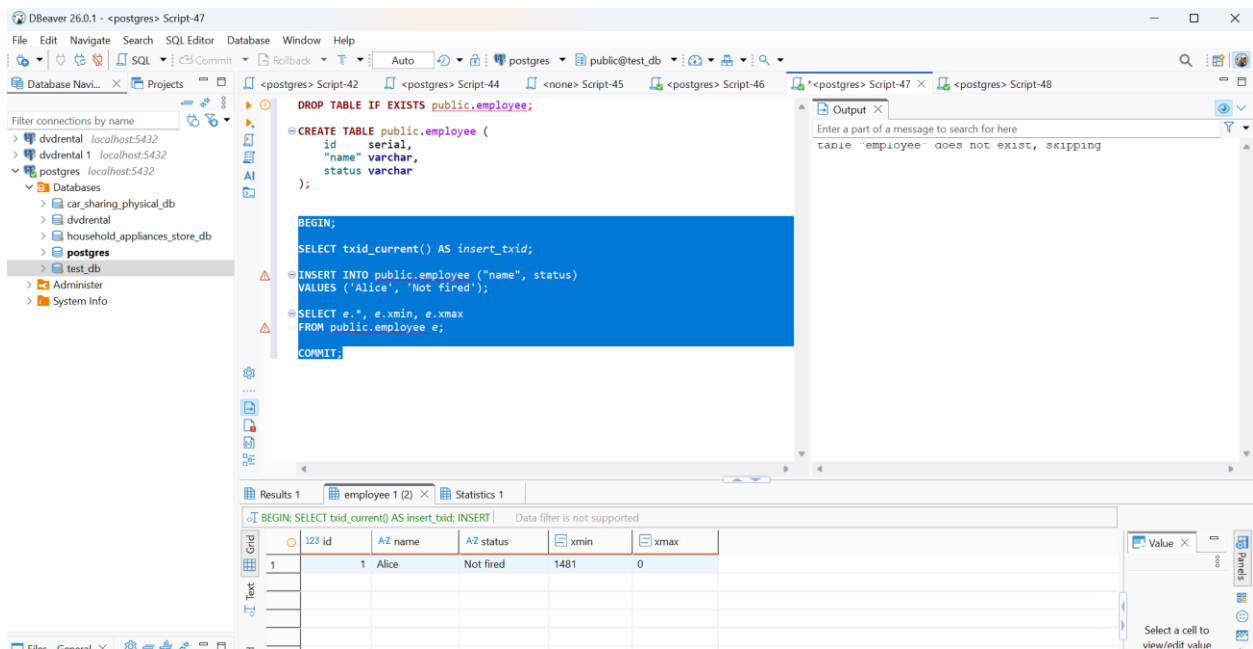
```
DROP TABLE IF EXISTS public.employee;  
  
CREATE TABLE public.employee (  
    id    serial,  
    "name" varchar,  
    status varchar  
);
```

2. Recreate the examples

first transaction

Session 1

```
BEGIN;  
  
SELECT txid_current() AS insert_txid;  
  
INSERT INTO public.employee ("name", status)  
VALUES ('Alice', 'Not fired');  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```

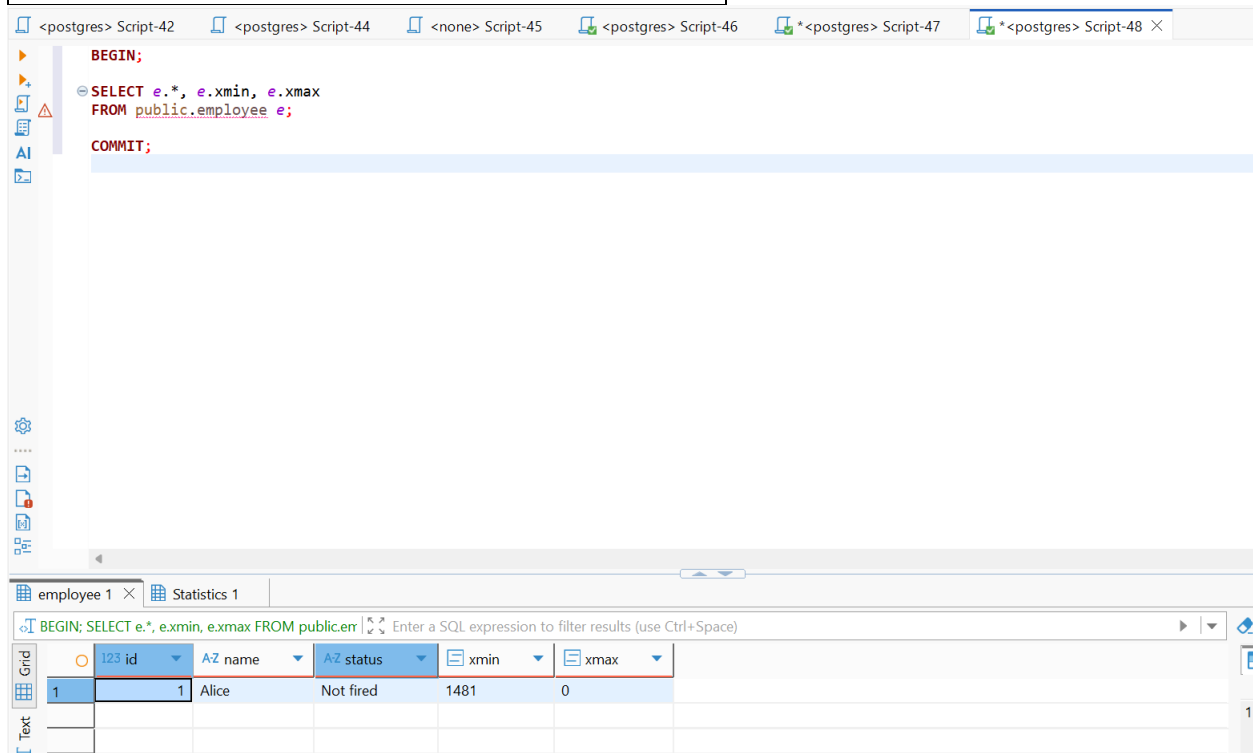


Session 2

BEGIN;

SELECT e.*, e.xmin, e.xmax
FROM public.employee e;

COMMIT;



Second transaction

Session 1

```
BEGIN;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;
```

Session 2

```
BEGIN;  
  
SELECT txid_current() AS delete_txid;  
  
DELETE FROM public.employee  
WHERE id = 1;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```

The screenshot shows a PostgreSQL IDE with a SQL script in the editor and its execution results in a table below.

SQL Script:

```
BEGIN;  
SELECT txid_current() AS delete_txid;  
DELETE FROM public.employee  
WHERE id = 1;  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
COMMIT;
```

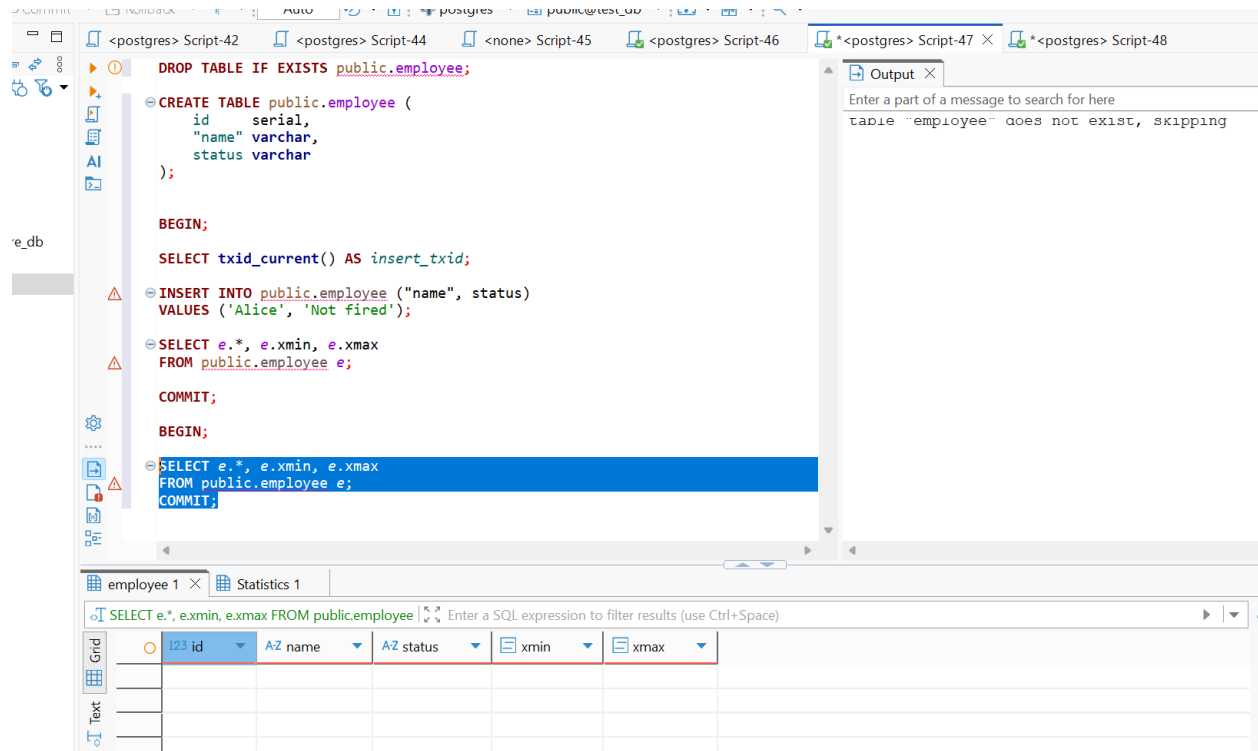
Execution Results:

The results pane shows a table with 5 columns: id, name, status, xmin, and xmax. The first row contains the value 123 in the id column. The table is currently empty of data.

id	name	status	xmin	xmax
123				

Return to Session 1 -

```
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```



after Session 2 commits, Session 1's second SELECT will no longer see Alice

```
INSERT INTO public.employee ("name", status)  
VALUES ('Alice', 'Not fired');
```

Third transaction: update Alice

Session 1:

```
BEGIN;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;
```

The screenshot shows a PostgreSQL IDE with a script editor on the left and an output window on the right. The script editor contains the following SQL commands:

```
CREATE TABLE public.employee (  
    id serial,  
    "name" varchar,  
    status varchar  
);  
  
BEGIN;  
  
SELECT txid_current() AS insert_txid;  
  
INSERT INTO public.employee ("name", status)  
VALUES ('Alice', 'Not fired');  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;  
  
BEGIN;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
COMMIT;  
  
INSERT INTO public.employee ("name", status)  
VALUES ('Alice', 'Not fired');
```

The output window on the right shows the following message:

```
Enter a part of a message to search for here  
table "employee" does not exist, skipping
```

Below the script editor, there is a table viewer showing the results of the query. The table has columns: id, name, status, xmin, and xmax. The data is as follows:

id	name	status	xmin	xmax
1	Alice	Not fired	1483	0

Session 2

```
BEGIN;  
  
SELECT txid_current() AS update_txid;  
  
UPDATE public.employee  
SET status = 'Fired'  
WHERE id = 2;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```

```
BEGIN;  
SELECT txid_current() AS delete_txid;  
  
DELETE FROM public.employee  
WHERE id = 1;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;  
  
BEGIN;  
SELECT txid_current() AS update_txid;  
  
UPDATE public.employee  
SET status = 'Fired'  
WHERE id = 2;  
  
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```

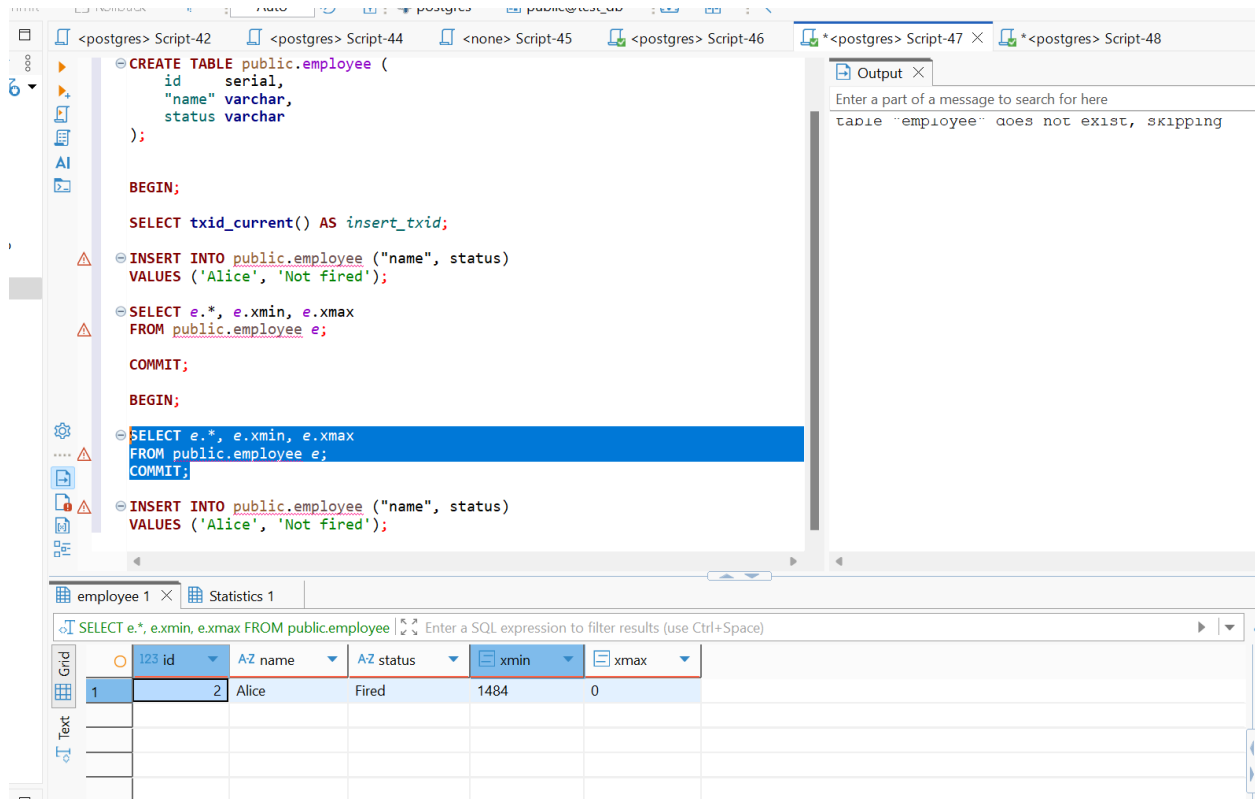
Results 1 | employee 1 (2) | Statistics 1

SQL: BEGIN; SELECT txid_current() AS update_txid; UPDA | Data filter is not supported

id	name	status	xmin	xmax
1	Alice	Fired	1484	0

Return to Session 1

```
SELECT e.*, e.xmin, e.xmax  
FROM public.employee e;  
  
COMMIT;
```



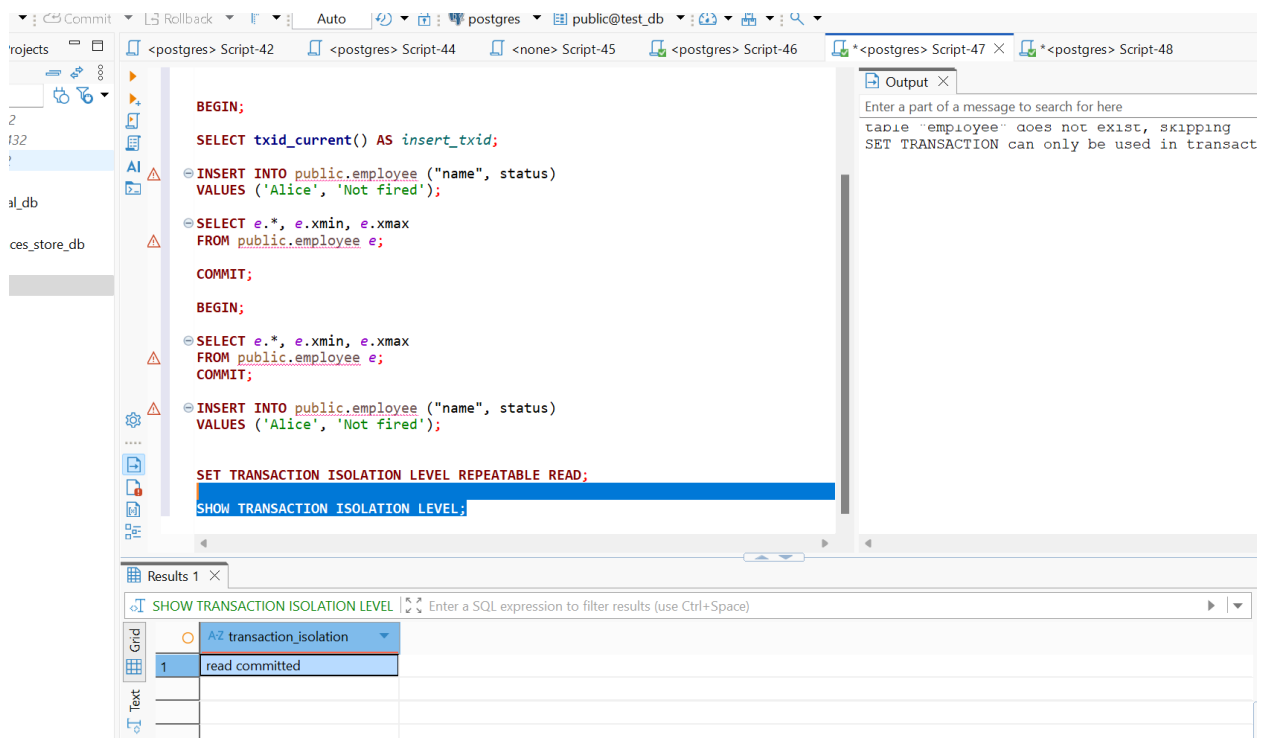
the second SELECT in Session 1 sees the committed update.

3. Set transaction isolation level to REPEATABLE READ

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

4. Check your current isolation level in each session with show transaction isolation level.

```
SHOW TRANSACTION ISOLATION LEVEL;
```

5. Recreate employee table and redo the second task

but modify the code so that select statements would now include cmin and cmax system columns. What changed?

```
DROP TABLE IF EXISTS public.employee;
```

```
CREATE TABLE public.employee (
  id serial,
  "name" varchar,
  status varchar
);
```

```
BEGIN;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT txid_current();
```

```
INSERT INTO public.employee ("name", status)
VALUES ('Alice', 'Not fired');
```

```
SELECT e.*, e.xmin, e.xmax, e.cmin, e.cmax
```

```
FROM public.employee e  
ORDER BY e.id;
```

```
COMMIT;
```

The screenshot shows a PostgreSQL IDE with a SQL script in the editor and its results in the output pane. The script is as follows:

```
SHOW TRANSACTION ISOLATION LEVEL;  
  
-----Task5-----  
  
DROP TABLE IF EXISTS public.employee;  
  
CREATE TABLE public.employee (  
    id serial,  
    "name" varchar,  
    status varchar  
);  
  
BEGIN;  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
SELECT txid_current();  
INSERT INTO public.employee ("name", status)  
VALUES ('Alice', 'Not fired');  
SELECT e.*, e.xmin, e.xmax, e.cmin, e.cmax  
FROM public.employee e  
ORDER BY e.id;  
COMMIT;
```

The output pane shows the following message:

```
table "employee" does not exist, skipping  
SET TRANSACTION can only be used in transaction blocks
```

The results pane shows the following table:

	id	name	status	xmin	xmax	cmin	cmax
1	1	Alice	Not fired	1486	0	0	0

cmin — command id inside the inserting transaction.

cmax — command id inside the deleting/updating transaction, or 0 if not deleted.

Under REPEATABLE READ, repeated SELECTs inside the same transaction see the same snapshot. So if another session deletes or updates a row, the open transaction may still see the old version.

6. cause a serialization anomaly on the employee table

Prepare data

```
TRUNCATE public.employee RESTART IDENTITY;
```

```
INSERT INTO public.employee ("name", status)  
VALUES  
    ('Alice', 'Not fired'),  
    ('Bob', 'Not fired');
```

Session 1

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT COUNT(*) AS not_fired_count  
FROM public.employee  
WHERE status = 'Not fired';
```

-- returns 2, so application decides it is safe to fire Alice

```
UPDATE public.employee  
SET status = 'Fired'  
WHERE "name" = 'Alice';
```

-- do not commit yet

The screenshot shows a PostgreSQL SQL Editor window with a script containing the following SQL statements:

```
FROM public.employee e  
ORDER BY e.id;  
  
COMMIT;  
  
TRUNCATE public.employee RESTART IDENTITY;  
  
INSERT INTO public.employee ("name", status)  
VALUES  
('Alice', 'Not fired'),  
('Bob', 'Not fired');  
  
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  
SELECT COUNT(*) AS not_fired_count  
FROM public.employee  
WHERE status = 'Not fired';  
  
-- returns 2, so application decides it is safe to fire Alice  
  
UPDATE public.employee  
SET status = 'Fired'  
WHERE "name" = 'Alice';  
  
-- do not commit yet
```

The results pane at the bottom shows the output of the SELECT statement:

not_fired_count
2

Session 2

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

```
SELECT COUNT(*) AS not_fired_count  
FROM public.employee  
WHERE status = 'Not fired';
```

-- also returns 2, so application decides it is safe to fire Bob

```
UPDATE public.employee
SET status = 'Fired'
WHERE "name" = 'Bob';

COMMIT;
```

The screenshot shows a PostgreSQL IDE interface. The top pane displays several SQL scripts. Script-42 contains an UPDATE statement. Script-44 contains an UPDATE statement with a WHERE clause on 'id'. Script-45 contains a SELECT statement. Script-46 contains a COMMIT statement. Script-47 contains a BEGIN ISOLATION LEVEL REPEATABLE READ; statement. Script-48 contains a SELECT COUNT(*) AS not_fired_count FROM public.employee WHERE status = 'Not fired'; statement, followed by a comment and another UPDATE statement. The bottom pane shows the results of the query in Script-48. The results are displayed in a table with one row and one column, showing the value 2.

```
SELECT txid_current() AS update_txid;

UPDATE public.employee
SET status = 'Fired'
WHERE id = 2;

SELECT e.*, e.xmin, e.xmax
FROM public.employee e;

COMMIT;

-----Task5-----

BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT COUNT(*) AS not_fired_count
FROM public.employee
WHERE status = 'Not fired';

-- also returns 2, so application decides it is safe to fire Bob

UPDATE public.employee
SET status = 'Fired'
WHERE "name" = 'Bob';

COMMIT;
```

Results 1 x Statistics 1

BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT | Enter a SQL expression to filter results (use Ctrl+Space)

not_fired_count
2

Return to Session 1

```
COMMIT;
SELECT *
FROM public.employee
ORDER BY id;
```

The screenshot shows a PostgreSQL IDE with a script editor and an output window. The script in the editor is as follows:

```

INSERT INTO public.employee ("name", status)
VALUES
('Alice', 'Not fired'),
('Bob', 'Not fired');

BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT COUNT(*) AS not_fired_count
FROM public.employee
WHERE status = 'Not fired';

-- returns 2, so application decides it is safe to fire Alice

UPDATE public.employee
SET status = 'Fired'
WHERE "name" = 'Alice';

COMMIT;
SELECT *
FROM public.employee
ORDER BY id;

```

The output window shows an error message: "table 'employee' does not exist, skipping SET TRANSACTION can only be used in transaction".

Below the script editor, a table view for 'employee 1' is shown with the following data:

id	name	status
1	Alice	Fired
2	Bob	Fired

7. Set isolation level to read committed

Try to cause a lost update database anomaly on the employee table (add more data if necessary).

Session 1

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

```
SHOW transaction_isolation;
```

```
UPDATE public.employee
SET name = CONCAT(name, ' ', '!')
WHERE id = 1;
```

Session 2

```
BEGIN ISOLATION LEVEL READ COMMITTED;
```

```
SHOW transaction_isolation;
```

```
UPDATE public.employee
```

```
SET salary = name = CONCAT(name, ' ', '!!')
WHERE id = 1;
```

-- this blocks until Session 1 commits

Session 1: Commit;

Session 2: Commit;

Check Results:

```
SELECT id, "name"
FROM public.employee
WHERE id = 1;
```

The screenshot shows a PostgreSQL IDE with a transaction in progress. The SQL editor contains the following code:

```
WHERE status = 'Not fired';
-- returns 2, so application decides it is safe to fire Alice
UPDATE public.employee
SET status = 'Fired'
WHERE "name" = 'Alice';

COMMIT;
SELECT *
FROM public.employee
ORDER BY id;

rollback
BEGIN ISOLATION LEVEL READ COMMITTED;
SHOW transaction_isolation;

UPDATE public.employee
SET name = CONCAT(name, ' ', '!!')
WHERE id = 1;

SELECT id, "name"
FROM public.employee
WHERE id = 1;
```

The output window shows the following message:

```
Enter a part of a message to search for here
table "employee" does not exist, skipping
SET TRANSACTION can only be used in transactio
there is no transaction in progress
there is no transaction in progress
```

The results window shows the following table:

id	name
1	Alice

Conclusions:

- Concurrent updates may block each other.
- Long transactions can reduce performance.
- Deadlocks are possible if transactions lock rows in different order.
- Applications may need retry logic.